Architecture choice

Abstraction: identify the structures that matter

Stake helps people complete goals through social accountability. A member sets a goal, stakes a private embarrassing photo, and submits photo check-ins. If the member misses the required completion window, the system publishes the stake to the permitted social feed.

Selected pattern: layered client-server

The **layered pattern** organizes responsibilities into presentation, application/business logic, and data access. The **client-server pattern** describes distribution: browsers on multiple machines send requests to one backend that manages shared data. These patterns describe complementary aspects of the same system.

The proposed implementation uses a **modular monolithic backend**: account, goal, social, and moderation modules run within one Express/Node application. They are logical components, not independently deployed microservices. React runs in the browser, and PostgreSQL runs as the database service.

Why this fits Stake

Separation of concerns: the interface can change without rewriting deadline rules or SQL queries. Each layer exposes a focused interface to the layer above it.

Shared, consistent state: all clients use the same server rules and database. Transactions help keep check-ins, missed windows, and released stakes consistent. The lecture recommends a shared database for structured data updated by multiple features.

Manageable scope: one backend is simpler for a three-person team to build, test, and deploy. Microservices would add service communication and deployment work without a demonstrated need for independent scaling.

Tradeoffs and design status

A backend failure can affect the whole application, and backend modules are deployed together. Clear module boundaries and tests limit coupling.

Architecture model

Internal layers, components, communication, and storage

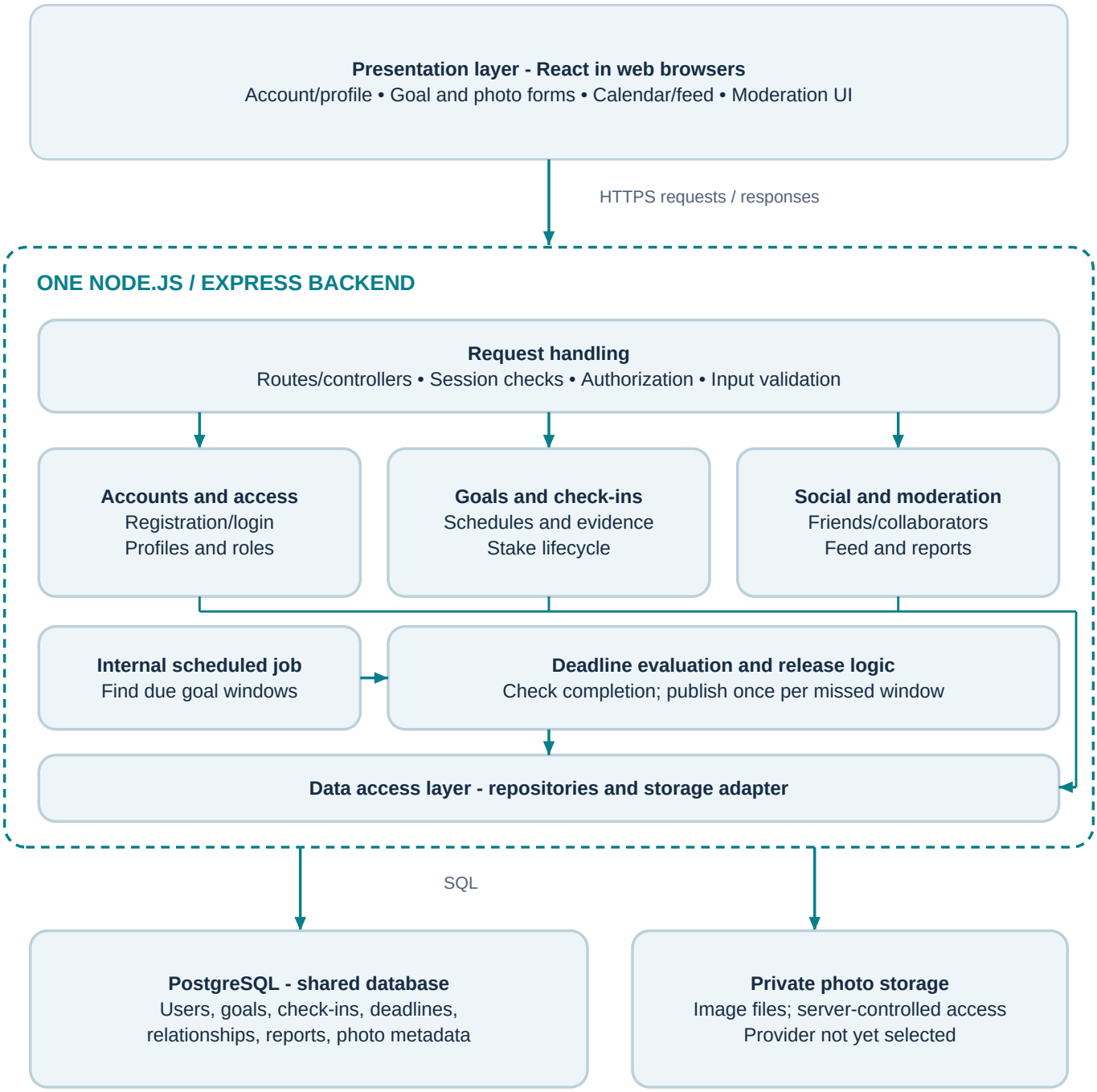

Figure 1. Arrows show calls or data access; responses return on the same connection. Business modules use the data access layer. The internal timer invokes goal logic without a browser request. Photo storage is a proposed application-owned resource, not a selected third-party integration.

System context model

External actors and the boundary of the complete Stake system

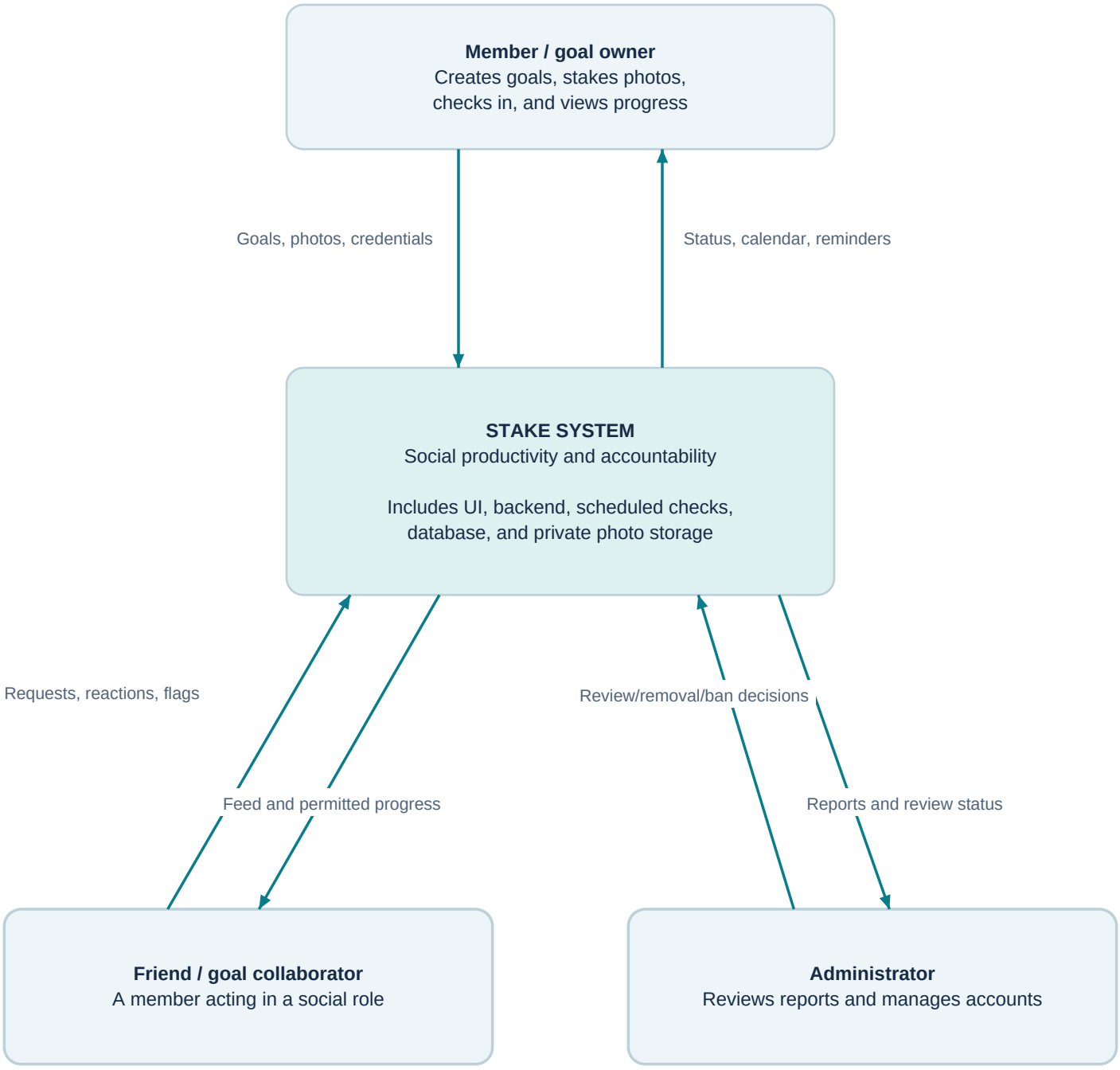

Figure 2. Each actor interacts through a browser. Friends/collaborators are members, not a third authorization role. The entire product is the system boundary, so its database and timer are internal. No external service integration is confirmed.

Component and data design

Decomposition: each component owns a focused responsibility

Presentation. React screens collect input and display server results. They do not decide whether a stake must be released.

Account and access. Register/login/logout, password verification, member/admin roles, and profile access. Authorization is enforced on server operations.

Goals, evidence, and stakes. Create schedules, accept check-in photos, calculate completion by goal window, keep stakes private, and restart a goal with a new stake.

Deadline processing. An internal recurring job finds due windows, rechecks completion, and calls the same goal rules used by request handlers. Release and feed-post creation occur in one database transaction.

Social and moderation. Manage friendships and goal collaborators, assemble authorized feed/calendar views, accept reports, and restrict review, removal, and banning to administrators.

Data access and image storage. Repositories handle SQL and transactions; a storage adapter handles private image files. PostgreSQL stores image identifiers and metadata. Browsers have no direct database connection.

Proposed relational data model

User owns many **Goals**; each goal has many **GoalWindows** and **Stakes**. Each window records its required count, cutoff, outcome, and assigned stake. **CheckIn** belongs to a window and references a photo; its submitting owner is obtained through the goal. A **Release** records a missed window whose stake was published.

Friendship relates two users and records request status. **GoalCollaborator** joins users to goals. **Photo** records owner and storage key. **FeedPost** references either a check-in or a release. **Report** references its reporter, the reported photo, and an optional reviewing administrator. A reported check-in is identified through its photo.

This relational structure avoids copying profile or image data into every feed item. Foreign keys preserve relationships; checks enforce exactly one FeedPost source, with a unique post per source. Unique collaboration and unordered friendship pairs prevent duplicates. Release.window_id is unique. Historical windows retain their original stakes. The application verifies that each window and its assigned stake belong to the same goal.

Example operation: submitting a check-in

React sends a photo and goal identifier to Express. The backend verifies identity, ownership, file validity, and the applicable deadline; stores the image privately; and commits the check-in and feed metadata. It returns the updated status. Authorized friends see the result when their feed refreshes.

Relational data model

Core lifecycle entities, keys, and relationship cardinalities

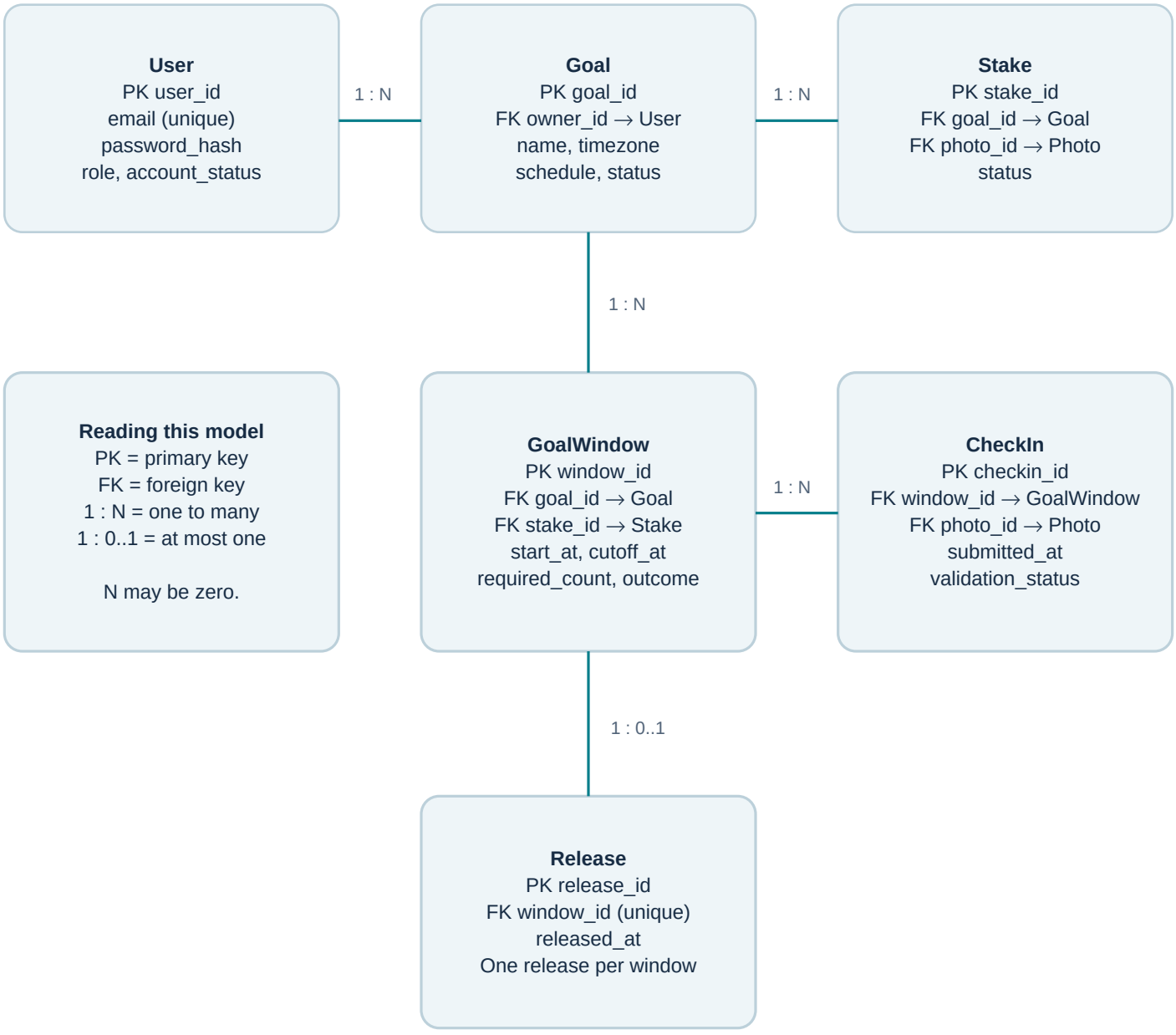

Figure 3. Each goal has an owner and many scheduled windows. Each window binds to one stake, receives check-ins, and may produce one release. FK labels identify references; selected core relationships are drawn to keep the diagram legible. Stake-to-window is also one-to-many.

Supporting entities: Photo(photo_id, owner_id, storage_key); Friendship(requester_id, recipient_id, status); GoalCollaborator(goal_id, user_id); FeedPost(post_id, checkin_id OR release_id); Report(report_id, reporter_id, photo_id, status, reviewer_id). Photo reports also identify suspect evidence through CheckIn.photo_id.

Technology and design decisions

Implementation choices and cross-cutting concerns

React. Reusable components suit repeated photo cards and goal/calendar views, reducing duplicated UI code. JavaScript handles interaction; HTML/CSS provide structure and responsive styling.

Express and Node.js. A shared JavaScript language reduces language switching for the team. Express routes expose a central API, so all clients use the same authorization and deadline rules. JSON carries structured data; multipart requests carry photos. Use HTTPS in deployment; no external API is assumed.

PostgreSQL. Related users, goals, and check-ins suit a relational schema. Constraints and transactions support consistent releases. One shared database avoids synchronizing separate client copies; use a local instance for development.

GitHub and Jira. GitHub provides version history and review to coordinate three contributors. Jira connects assigned backlog work to priorities and estimates. Both support development and remain outside the runtime context model.

Cross-cutting concerns

Security: hash passwords; validate uploads; check authorization for goals, reports, and photos; keep unreleased stakes inaccessible to other members; limit database credentials. Members report content, while administrators decide removals and bans.

Reliability: persist goal windows, store cutoff instants in UTC with the goal timezone, and recheck due windows after a restart. Serialize check-in acceptance and deadline evaluation for the same window so the outcome uses committed evidence; enforce one release per window. Exact grace and late-upload rules require agreement.

Performance and maintainability: index deadlines and feed relationships, paginate feeds, constrain image sizes, and separate request handlers, business rules, and data access. These concerns span all layers, as described in the lecture.